

OpenGL

Grupo

Nomes	Matrícula	Curso
Beatriz Vaz Pedroso Santos Cobral	2082	GEC
Felipe Silva Loschi	601	GES
Felipe Tagawa Reis	2037	GEC
Pedro Henrique Ribeiro Dias	529	GES

Conceituação



Figura 1: OpenGL

O **OpenGL (Open Graphics Library)** é uma **API (Application Programming Interface)** de código aberto e multiplataforma, consolidada como o padrão da indústria para o desenvolvimento de aplicações gráficas 2D e 3D, jogos e simulações.

História

A criação do OpenGL foi uma resposta direta ao cenário caótico da década de 80, quando a falta de padronização obrigava fabricantes de hardware a utilizarem conjuntos de instruções proprietários. Esse isolamento tecnológico dificultava o desenvolvimento de aplicações multiplataforma, exigia esforços redundantes das empresas e limitava a compatibilidade no mercado.

Nesse contexto, a Silicon Graphics Inc. (SGI) destacou-se com o IRIS GL, uma API considerada o estado da arte por sua facilidade de uso. Para consolidar sua tecnologia como padrão global e enfrentar a concorrência de gigantes como IBM e Sun Microsystems, a SGI decidiu abrir sua tecnologia. Como o IRIS GL continha códigos proprietários e funções alheias aos gráficos (como controle de periféricos), a empresa refinou a ferramenta, removendo excessos e lançando, em 1992, o OpenGL. Desde então, a API é gerida pelo Architecture Review Board (ARB), um conselho formado pelas maiores potências do setor, como NVIDIA, Intel e HP.

O sucesso duradouro do OpenGL fundamenta-se em sua arquitetura flexível, que permite a inclusão de extensões para novos recursos de hardware e a emulação via software para garantir que aplicações rodem mesmo em equipamentos menos potentes. Embora tenha enfrentado a forte concorrência do Direct3D (DirectX) da Microsoft — que se tornou dominante no Windows devido ao foco em jogos e efeitos visuais —, o OpenGL mantém sua relevância pela versatilidade. Atualmente, sua capacidade de adaptação é seu maior trunfo, permitindo que desenvolvedores portem jogos e softwares entre PCs, consoles e dispositivos móveis com uma eficiência que APIs restritas a um único sistema operacional não conseguem oferecer.

Dispositivos Gráficos

Dispositivos de Processamento

A interação mais profunda ocorre entre o OpenGL e os dispositivos de processamento. O OpenGL atua como uma interface (API) que faz a ponte entre o software e o hardware de processamento gráfico.

- GPU: O OpenGL trabalha no modo "imediato", evitando comandos diretamente para a GPU. É dentro da GPU que ocorre o Pipeline Gráfico (processamento de vértices, rasterização e processamento de fragmentos) para gerar a imagem final com alto desempenho através de arquitetura paralela;
- CPU: A CPU fica responsável por executar o código da aplicação em uma linguagem de alto nível e fazer as chamadas de função do OpenGL que serão interpretadas e enviadas para o hardware gráfico.

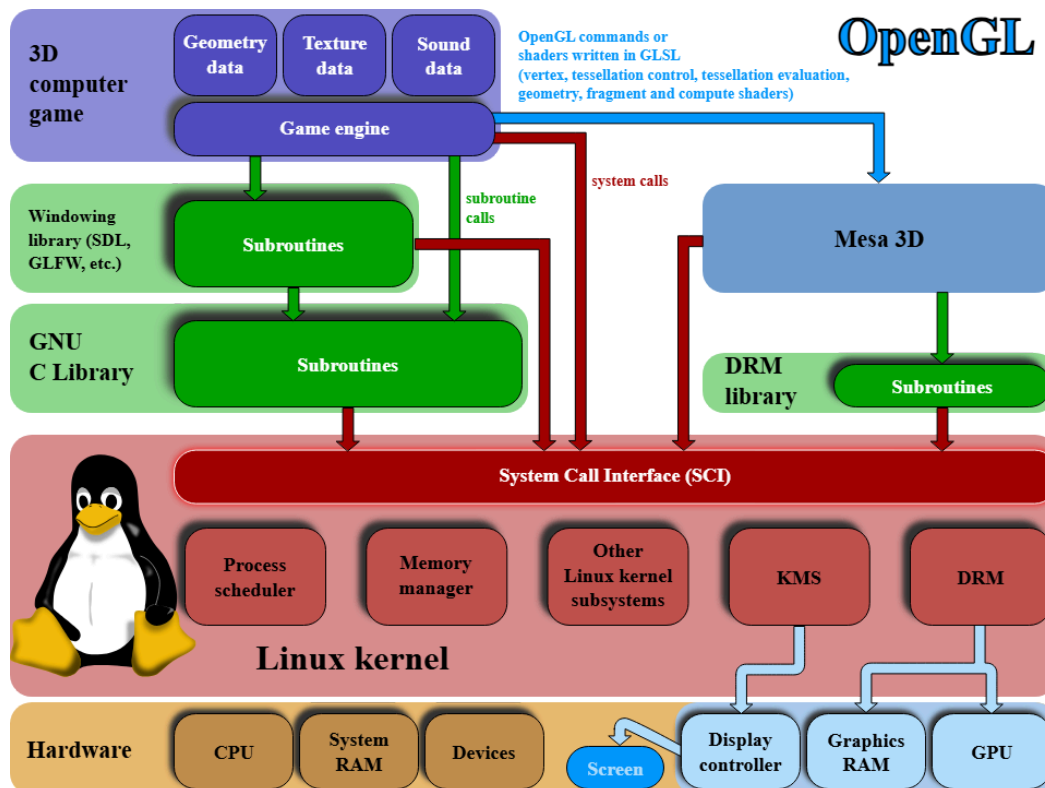


Figura 2: Arquitetura de Processamento. A CPU gerencia a lógica da aplicação e chamadas de API, enquanto a GPU executa o pipeline gráfico em paralelo para processamento massivo de dados.

Dispositivos de Entrada

A fim de manter as características "generalização" e "independência de plataforma", o OpenGL **não fornece recursos de entrada**. Ele não consegue, por si só, detectar coordenadas de mouse ou pressões de teclas, sendo necessário um kit de ferramentas adicional ou um ambiente de navegador (HTML/JavaScript) para capturar esses eventos.

Dispositivos de Saída

O OpenGL é responsável por calcular o que deve ser exibido, mas ele não controla a "janela" física onde isso aparece.

- **Framebuffer:** O objetivo final do OpenGL é gerar pixels (posição e cor) no *framebuffer*. O *framebuffer* é a memória que armazena a imagem que será lida pelo display;
- **Independência de Janela:** O OpenGL não oferece recursos para criar ou redimensionar janelas. Ele depende do sistema operacional ou de bibliotecas auxiliares para preparar o display onde o resultado da renderização será mostrado.

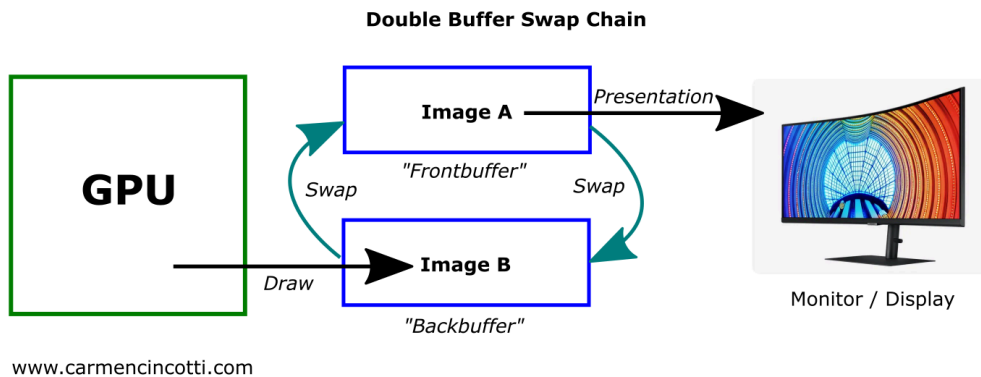


Figura 3: Estágios finais do Pipeline Gráfico. Os fragmentos processados são combinados e escritos no Framebuffer, onde as posições e cores são armazenadas antes de serem enviadas para o dispositivo de exibição (Display).

Outros Dispositivos

Uma vez que a ideia principal dessa ferramenta seja a geração de imagens a partir de modelos geométricos, dispositivos de captura (câmeras/scanners) ou de saída física (impressoras) estariam **fora do escopo funcional da API**, sendo tratados pelo sistema operacional ou por outras APIs específicas.

Transformações Geométricas

No OpenGL moderno, as transformações geométricas não são realizadas por funções internas fixas, mas sim através de matrizes calculadas na CPU e enviadas para a GPU. A biblioteca padrão para lidar com essa matemática de forma compatível com o OpenGL é a **GLM (OpenGL Mathematics)**.

Configurações da Biblioteca GLM

A GLM é uma biblioteca *header-only* (apenas cabeçalhos), o que significa que não precisa de compilação ou linkagem separada; basta incluir os arquivos no projeto.

Implementação de Transformações na CPU

Para transformar um objeto, primeiro cria-se uma **matriz de identidade** (uma matriz que não altera o objeto) e, em seguida, aplicam-se as operações desejadas.

Transformações Básicas

Para criar uma matriz de transformação, você sempre começa com uma **matriz identidade (1.0 na diagonal)** e aplica as funções da GLM.

Operação	Função GLM	Descrição Prática
Escala	<code>glm::scale(m, vec3(s))</code>	Altera o tamanho. Valores > 1 aumentam; entre 0 e 1 diminuem.
Rotação	<code>glm::rotate(m, rad, axis)</code>	Gira o objeto em torno de um eixo. O ângulo

		deve ser em radianos.
Translação	<code>glm::translate(m, vec3(t))</code>	Mova o objeto para uma nova posição \$(x, y, z)\$.

Transformações Avançadas: Cisalhamento e Reflexão

Embora menos comuns, o OpenGL suporta transformações que deformam ou espelham a geometria.

Reflexão

Obtida usando a função de escala com valores negativos

`glm::scale(model, glm::vec3(-1.0f, 1.0f, 1.0f));` espelha o objeto no eixo X.

Cisalhamento

Inclina o objeto, transformando um retângulo em um paralelogramo.

- Pode ser feito via extensão: `#include <glm/gtx/transform2.hpp>` e a função `glm::shearX3D(m, y_factor, z_factor)`.
- Ou manualmente, alterando valores fora da diagonal: `matrix[12] = factor;`.

Ordem das Matrizes

No OpenGL, as transformações são lidas de cima para baixo no código, mas aplicadas da **direita para a esquerda** no vértice.

Ordem Recomendada no código

1. `glm::translate` (Última a ser aplicada, coloca o objeto no lugar certo).
2. `glm::rotate` (Segunda a ser aplicada, gira o objeto sobre seu centro).
3. `glm::scale` (Primeira a ser aplicada, define o tamanho antes de mover).

Caminho do Vértice

Para levar um objeto do software de modelagem até a tela, ele passa por três matrizes principais chamadas de sequência **MVP**.

1. **Model Matrix:** Move o objeto do seu espaço local para o mundo global (posicionamento).
2. **View Matrix:** Simula a câmera. Em vez de mover a câmera, o OpenGL move o mundo inteiro na direção oposta (usando `glm::lookAt`).
3. **Projection Matrix:** Define a perspectiva (objetos distantes ficam menores) ou ortografia (tamanho fixo, útil para 2D).

A fórmula final no shader é:

$$V_{final} = P_{projection} \cdot V_{view} \cdot M_{model} \cdot V_{local}$$

Integração com Shaders

Após calcular a matriz final na CPU, você deve enviá-la para o **Vertex Shader**.

Detalhes Técnicos Críticos

- **Layout de Memória (Column-Major):** O OpenGL armazena matrizes por colunas, não por linhas. O GLM segue esse padrão nativamente, por isso o parâmetro `transpose` em `glUniformMatrix4fv` é quase sempre `GL_FALSE`.

- **Z-Fighting:** Devido à precisão finita do hardware, objetos muito próximos e distantes da câmera podem "piscar" (lutar pela frente). Uma solução comum é manter o plano *near* da projeção o mais longe possível de 0.
- **Diferença entre Pontos e Direções:** Ao transformar normais de superfície, se houver escala não-uniforme, deve-se usar a "Matriz Normal" (inversa da transposta da matriz de modelo) para evitar que a iluminação fique distorcida.

Este sistema modular permite que o OpenGL processe milhões de vértices por segundo, delegando a complexidade geométrica para a GPU enquanto o desenvolvedor mantém o controle lógico através de matrizes na CPU.

Coors

No OpenGL, o gerenciamento de cores é baseado no modelo RGB, podendo incluir o canal alpha. Funciona como um estado de máquina: uma vez definida uma cor, todos os objetos desenhados posteriormente a ela terão essa cor, até que haja uma alteração.

Quando uma fonte de luz é definida no OpenGL, é necessário definir uma cor a esta fonte de luz. Se uma cor de uma fonte de luz for multiplicada pela cor de um objeto, a cor resultante seria a cor refletida do objeto (e, portanto, sua cor percebida).



Figura 4: Absorção e Reflexão da Luz

Especificações de Valores

A função principal utilizada é a `gl.glColor3f(r, g, b);`

- **O sufixo "3f":** Indica que a função recebe 3 parâmetros do tipo **float**.
- **Escala:** Os valores devem estar no intervalo entre **0.0 (0%)** e **1.0 (100%)**.

```

glBegin(GL_QUADS)
glColor3f(1.0, 0.0, 0.0) // Red color
glVertex2i(-50, -50)
glColor3f(0.0, 1.0, 0.0) // Green color
glVertex2i( 50, -50)
glColor3f(0.0, 0.0, 1.0) // Blue color
glVertex2i( 50, 50)
glColor3f(1.0, 1.0, 0.0) // Yellow color
glVertex2i(-50, 50)
glEnd()

```

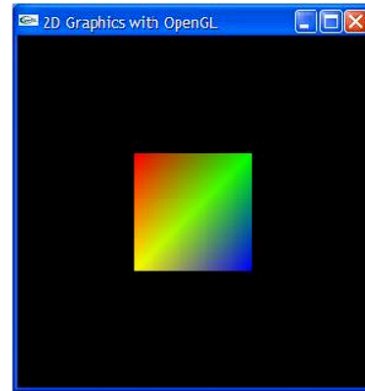


Figura 5: Manipulação de Cores em um objeto quadrado utilizando funções do OpenGL.

Dois Objetos em Cena

A cena precisa de dois atores distintos. A **fonte de luz** (cubo pequeno) usa um shader separado que sempre emite branco puro — ela não pode ser afetada pela própria iluminação que gera. O **objeto iluminado** usa o `lightingShader`, que recebe `objectColor` e `lightColor` como uniforms e os multiplica no fragment shader.



Figura 6: Objeto Iluminado versus Fonte de Luz

Cada objeto tem seu próprio pipeline. No shader do objeto, o vertex shader transforma as posições e o fragment shader faz a multiplicação de cores. No shader da lâmpada, o fragment shader ignora qualquer uniform de cor e simplesmente retorna `vec4(1.0)` — branco puro — porque a lâmpada precisa parecer que *emite* luz, não que a *recebe*.

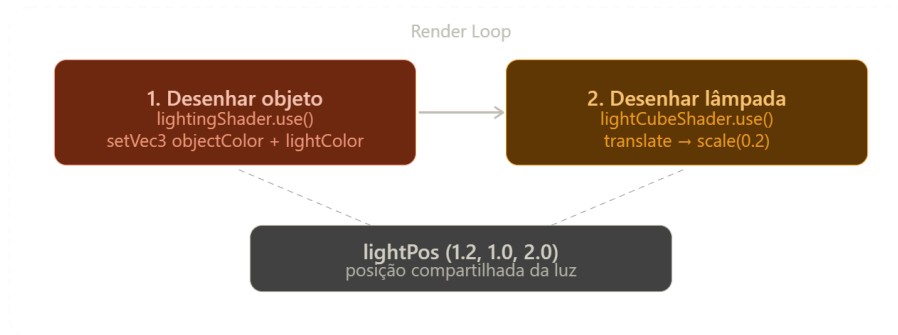


Figura 7: Processo de renderização

Cores e Vértices

A aplicação da cor depende de **onde** a função é chamada em relação ao bloco de desenho:

- **Cor Fixa (Flat Shading)** : Se for chamada `glColor` antes do `glBegin()` , o objeto inteiro terá uma cor única;
- **Gradiente (Smooth Shading)** : Se for chamada `glColor` para cada vértice dentro do bloco `glBegin()` / `glEnd()` , o OpenGL realiza a **interpolação linear** (podendo criar degradês, como visto na figura 4);
- Outro ponto crucial é o **Modelo de Sombreamento** (`glShadeModel`): o modo *Smooth* (padrão) realiza a interpolação entre vértices, enquanto o modo *Flat* desabilita o degradê, aplicando uma cor sólida baseada no último vértice da primitiva.

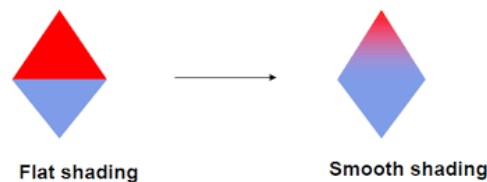


Figura 8: Diferenças entre Flat e Smooth Shading.

Limpeza do Buffer

Permite que a animação aconteça sem que os quadros se sobreponham, criando a ilusão de movimento ou permitindo que uma nova imagem seja desenhada do zero.

Se um quadrado for desenhado na posição $x=1$ e, no próximo quadro, o mesmo quadrado na posição $x=2$ **sem limpar o buffer**, dois quadrados poderão ser vistos (ou pelo menos um rastro).

- A "**limpeza**" apaga a lousa inteira com uma cor sólida para que o novo desenho seja a única coisa visível;
- Isso é essencial para **jogos e simulações**, onde os objetos mudam de lugar a cada milissegundo.

O papel do `glClearColor`

Antes de mandar limpar, a seguinte pergunta deve ser feita ao OpenGL: "Se tudo for apagado, que cor deve ficar no fundo?".

- `glClearColor(0.0f, 0.0f, 0.0f, 1.0f);` → Define que o fundo será **Preto**.
- `glClear(GL_COLOR_BUFFER_BIT);` → Executa a limpeza de fato.

Double Buffering

Na computação gráfica moderna usam-se dois buffers:

1. **Front Buffer**: A imagem que está aparecendo no monitor.

2. **Back Buffer:** Onde o OpenGL está limpando e desenhando a "próxima" imagem em segredo. Quando o desenho no *Back Buffer* termina, eles trocam de lugar (*Swap*). Isso evita que o usuário veja a tela sendo "apagada" e "redesenhada", o que causaria um efeito de cintilação irritante.

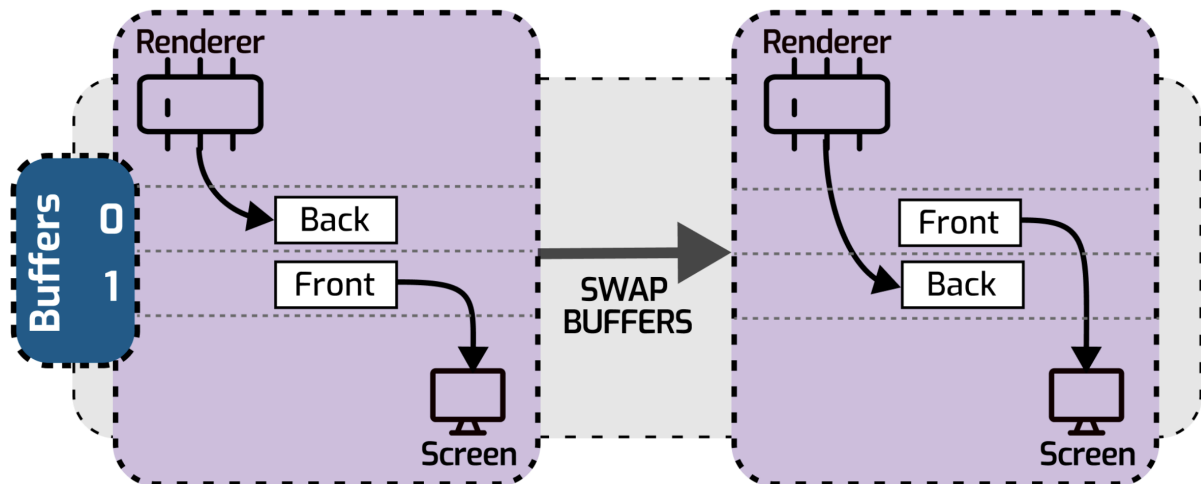


Figura 9: Mecanismo de Swap Chain. Demonstração da alternância entre buffers para ocultar o processo de atualização de pixels no Framebuffer, essencial para animações de alto desempenho.

Curvas e Superfícies

A representação de formas orgânicas e fluidas é um dos maiores desafios da Computação Gráfica. No **OpenGL moderno (versão 4.0+)**, a implementação de geometria curva evoluiu de funções estáticas para o **Pipeline de Tesselação**. Essa abordagem transfere a carga matemática da CPU para a GPU, permitindo que superfícies complexas sejam geradas dinamicamente com alta performance.

O Conceito de Patch: A Primitiva Abstrata

Diferente das primitivas clássicas como `GL_TRIANGLES`, que possuem topologia rígida, o OpenGL moderno introduz o **Patch**. Um patch é uma coleção abstrata de pontos de controle que não define uma forma visual imediata, mas sim a "casca" matemática que os shaders usarão para calcular a curvatura.

Para uma curva de Bézier cúbica, por exemplo, enviamos 4 pontos de controle. O código abaixo exemplifica a configuração dessa estrutura no lado da aplicação (C++):

```
// 1. Definição dos pontos de controle no espaço 3D
GLfloat controlPoints[] = {
    -1.0f, -1.0f, 0.0f, // P0
    -0.5f,  1.0f, 0.0f, // P1
    0.5f,  -1.0f, 0.0f, // P2
    1.0f,  1.0f, 0.0f  // P3
};

// 2. Configuração do estado do OpenGL para interpretar 4 vértices por patch
glPatchParameteri(GL_PATCH_VERTICES, 4);

// 3. Comando de desenho utilizando a primitiva de tesselação
glDrawArrays(GL_PATCHES, 0, 4);
```

Pontos de Controle: São as "âncoras" matemáticas. A curva é definida pela influência gravitacional de cada ponto.

Abstração: No OpenGL moderno, o GL_PATCHES não desenha linhas automáticas; ele envia estes 4 pontos para a GPU processar a curvatura via formulas matemáticas.

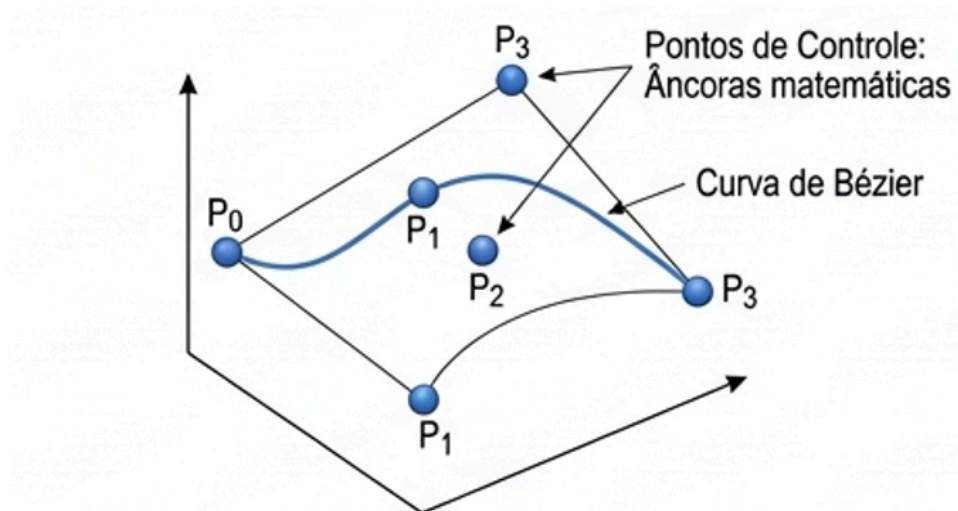


Figura 10: O Conceito de Patch e Pontos de Controle. Um conjunto de 4 pontos (P0, P1, P2, P3) flutuando no espaço 3D. Entre eles, uma linha elástica (a curva de Bézier) que é "puxada" pelos pontos internos (P1 e P2), mas não necessariamente passa por eles.

Tessellation Control Shader (TCS): Gerenciamento de Detalhes

O **TCS** atua como o gestor do **Nível de Detalhe (LOD - Level of Detail)**. Sua função principal é determinar o quão "suave" a curva deve ser, definindo em quantos segmentos a primitiva será subdividida.

No shader abaixo, configuramos a segmentação da curva (conhecida como *isolines*):

```
#version 400 core
layout (vertices = 4) out;

void main() {
    // Repassa a posição dos pontos de controle para o avaliador
    gl_out[gl_InvocationID].gl_Position = gl_in[gl_InvocationID].gl_Position;

    // Define a densidade da malha: quanto maior o valor, mais suave a curva
    if (gl_InvocationID == 0) {
        gl_TessLevelOuter[0] = 1;
        gl_TessLevelOuter[1] = 64; // A curva será dividida em 64 segmentos
    }
}
```

A Figura 5 demonstra que a geometria enviada pela CPU é a mesma (apenas os pontos de controle). A suavidade (densidade de triângulos) é decidida inteiramente dentro da placa de vídeo através do parâmetro `gl_TessLevelOuter`.

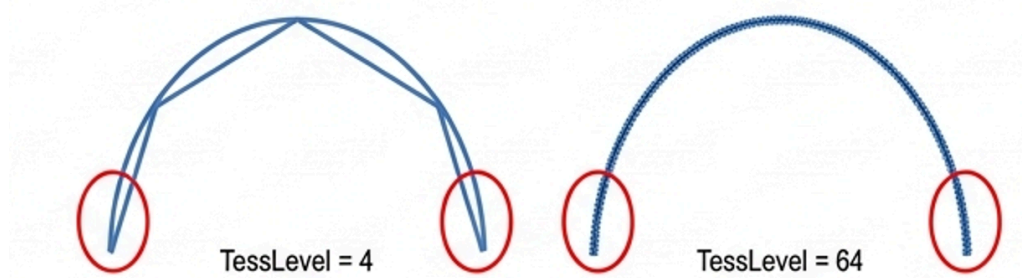


Figura 11: Níveis de Tesselação (LOD - Level of Detail). Uma comparação lado a lado de dois arcos.

O arco da esquerda é composto por apenas 4 segmentos retos (parecendo um trapézio).

O arco da direita é uma curva perfeitamente suave, composta por 64 pequenos segmentos.

Tessellation Evaluation Shader (TES): A Matemática da Forma

Após a subdivisão feita pelo hardware, o **TES** é executado para cada novo vértice gerado. É aqui que aplicamos as equações matemáticas (como os Polinômios de Bernstein para Bézier) para posicionar os vértices no espaço, transformando os pontos de controle em uma superfície visível.

TCS (Control): O shader que decide "o quanto" dividir.

TES (Evaluation): O shader que decide "onde" cada novo ponto fica, aplicando a equação de Bézier ou NURBS.

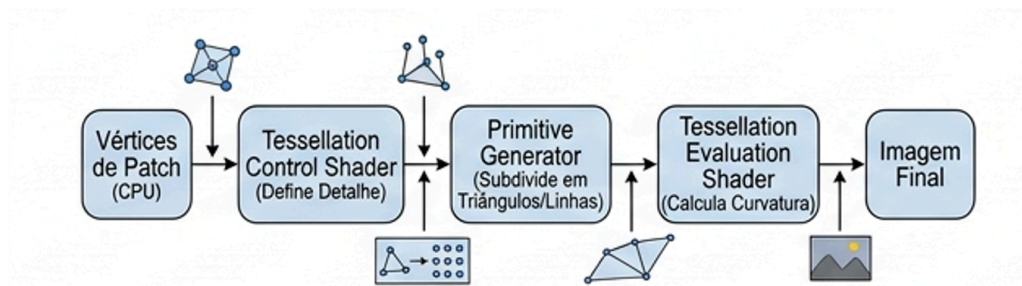


Figura 12: O Pipeline de Tesselação (TCS e TES)

De curvas para superfícies

A Figura 10 demonstra a aplicação prática da matemática de curvas (como Bézier ou NURBS) expandida para duas direções simultâneas: os parâmetros u e v . Em vez de uma linha elástica puxada por 4 pontos, agora temos uma 'manta' elástica puxada por uma **grade de 16 pontos de controle**.

Esta figura é crucial porque ela ilustra como os conceitos abstratos de álgebra e cálculo de superfícies paramétricas, que estudamos na teoria, são visualizados e aplicados na prática pela computação gráfica.

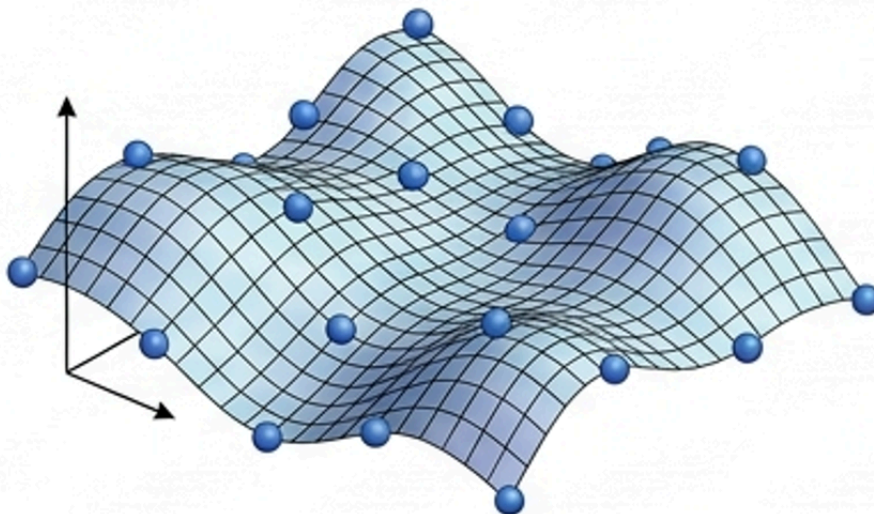


Figura 13: De Curvas para Superfícies (Patches 2D)

O OpenGL transforma uma coleção simples de pontos de controle em uma geometria suave, detalhada e pronta para receber iluminação e texturas.

O Equilíbrio entre Estética e Performance

Em suma, a implementação de **Curvas e Superfícies no OpenGL moderno** representa o equilíbrio ideal entre precisão matemática e eficiência computacional. Ao deixar de lado a renderização de malhas estáticas e pesadas em favor do **Pipeline de Tesselação**, a biblioteca permite que desenvolvedores criem mundos orgânicos e superfícies complexas com um custo de memória reduzido.

A **Suavidade Adaptativa** não é apenas um detalhe visual; é uma ferramenta de otimização crítica. Ela garante que a GPU processe apenas o detalhamento necessário para a percepção do usuário, mantendo silhuetas perfeitas em objetos próximos e economizando recursos em objetos distantes. Assim, o estudo das curvas paramétricas deixa de ser um conceito puramente teórico de cálculo para se tornar o alicerce da geometria fluida e do realismo nos motores gráficos contemporâneos.

Animação

OpenGL em si não é uma ferramenta de animação pronta. A animação acontece quando você redesenha a cena várias vezes (frame a frame), mudando posições, rotações, cores, etc.

Alguns exemplos de engines e frameworks que usam o OpenGL por baixo são Unity, Unreal Engine, Godot, Blender, Maya e 3ds Max.

Realismo e Iluminação

Visibilidade (Z-Buffer)

O algoritmo interno do OpenGL possui um **depth buffer** (buffer de profundidade) que faz uma associação de profundidade para cada pixel da janela. Geralmente, esses valores são iniciados com o maior valor de profundidade possível.

Para que a visibilidade funcione corretamente, precisamos:

1. Solicitar o buffer na inicialização: `glutInitDisplayMode(GLUT_DEPTH | ...)`.

2. Habilitar o teste: `glEnable(GL_DEPTH_TEST)` .

3. **Limpar o buffer a cada frame:** Dentro da função de desenho, devemos usar `glClear(GL_COLOR_BUFFER_BIT or GL_DEPTH_BUFFER_BIT)` .

Ao ser atribuído um valor de profundidade para um novo pixel, o OpenGL o compara com o valor já armazenado: se o pixel estiver mais próximo da câmera, ele "ganha" o teste, é pintado e o buffer atualizado. Caso esteja atrás, ele é desprezado, evitando que objetos distantes apareçam na frente de objetos próximos.

Nesse caso, o algoritmo utilizado para o cálculo da visibilidade é o **Z-Buffer**

Iluminação no OpenGL

No OpenGL, as cores das luzes serão determinadas pela porcentagem de vermelho, verde e azul que elas emitem, ou seja, são baseadas nos canais RGB, assim como o material de um objeto, que determina o que ele pode refletir. Dentro de uma cena, a luz pode vir de várias fontes, e estas, por sua vez, podem ser ligadas e desligadas individualmente.

No sistema do OpenGL, a luz apenas terá efeito caso exista um objeto (ou qualquer superfície) que absorva a luz emitida e a reflita. Sendo assim, os objetos podem ser compostos de "materiais" com diferentes propriedades, alguns podem emitir luz, outros podem refletir a luz absorvida em todas as direções e outros ainda podem refletir a luz em uma única direção de incidência

Tipos de Iluminação:

O OpenGL busca fazer sua iluminação a partir de aproximações fáceis de calcular do mundo real. Um desses modelos mais utilizados é o Phong, que consiste basicamente da combinação de três componentes:

- Luz ambiente
- Luz difusa
- Luz especular

vejamos cada tipo e sua aplicação a seguir.

Ambiente:

Uma luz que vem de todas as direções, resultado de uma iluminação ambiente. É uma luz que rebate em tantas superfícies que deixa o ambiente iluminado uniformemente.

Para inserirmos uma iluminação ambiente no OpenGL precisamos:

1. Ativação da iluminação

```
glEnable(GL_LIGHTING);
```

2. Definição da cor da luz ambiente

```
const
```

```
// Define a cor da luz ambiente
```

```
LuzAmbient : Array[0..3] of GLfloat = (1.0,1.0,1.0, 1.0);
```

Por padrão definimos essa cor como sendo branca (1,1,1). Pois assim teremos certeza de que todas as cores do objeto sejam visíveis e refletidas.

No caso de usarmos valores menores do que 1.0, a intensidade da cor será diminuída

3. Ativação do uso da luz ambiente

```
glLightModelfv(GL_LIGHT_MODEL_AMBIENT, @LuzAmbient);
```

```
// o primeiro parâmetro define o modelo de iluminação e o segundo os parâmetros
```

```
// deste modelo através de um vetor
```

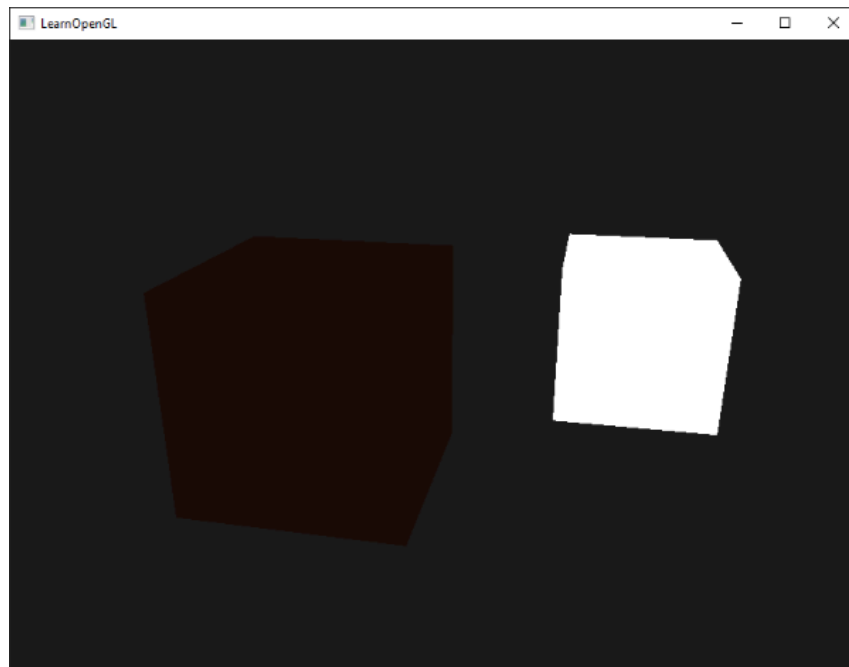


Figura 14: representação da luz ambiente

Difusa:

Diferentemente da luz ambiente, esta não vem de todas as direções, vindo apenas de uma. Porém, após atingir uma superfície ela é refletida em todas as direções, o que cria um efeito que faz parecer que o brilho do objeto é o mesmo independente da posição da câmera. É a componente que cria o efeito de volume e dar forma aos objetos.

- Ela é responsável por um efeito degradê nos objetos
- Precisamos determinar:
 - a componente ambiente da luz (opcional)
 - a cor dessa luz difusa dada pelos canais RGB
 - as coordenadas da posição da luz (x,y,z)

Para o OpenGL medir o ângulo de incidência do raio de luz, são utilizados vetores normais à superfície no ponto de incidência, dessa forma, o ângulo poderá ser facilmente calculado através do produto escalar entre os dois vetores.

Para inserimos uma luz difusa no OpenGL precisamos então:

1. Definir a cor da componente ambiente da luz, a posição e a cor da luz difusa (usando constantes)

```
LuzAmbiente : Array[0..3] of GLfloat = (0.5, 0.5, 0.5, 1.0);
// Cor da Luz Difusa
LuzDifusa : Array[0..3] of GLfloat = (0.8, 0.8, 0.8, 1.0);
// Posição da Luz Difusa
PosicaoLuz : Array[0..3] of GLfloat = (30.0, 30.0, 0.0, 1.0);
```

2. Ativar o uso da componente da luz ambiente e o color tracking

```
// Habilita o uso de iluminação
glEnable(GL_LIGHTING);
// Ativa o uso da luz ambiente
glLightModelfv(GL_LIGHT_MODEL_AMBIENT, @LuzAmbient);
// Ativa o "Color Tracking"
```

```
glEnable(GL_COLOR_MATERIAL);
```

2. Definir e ativar a luz difusa

```
// Define os parâmetros da Luz número Zero  
glLightfv(GL_LIGHT0, GL_AMBIENT, @LuzAmbiente);  
glLightfv(GL_LIGHT0, GL_DIFFUSE, @LuzDifusa );  
glLightfv(GL_LIGHT0, GL_POSITION, @PosicaoLuz );  
glEnable(GL_LIGHT0);  
// Exibe uma linha da posição da luz até (0,0,0)  
glBegin(GL_LINES);  
glVertex4fv(@PosicaoLuz);  
glVertex3f(0,0,0);
```

Exemplo:

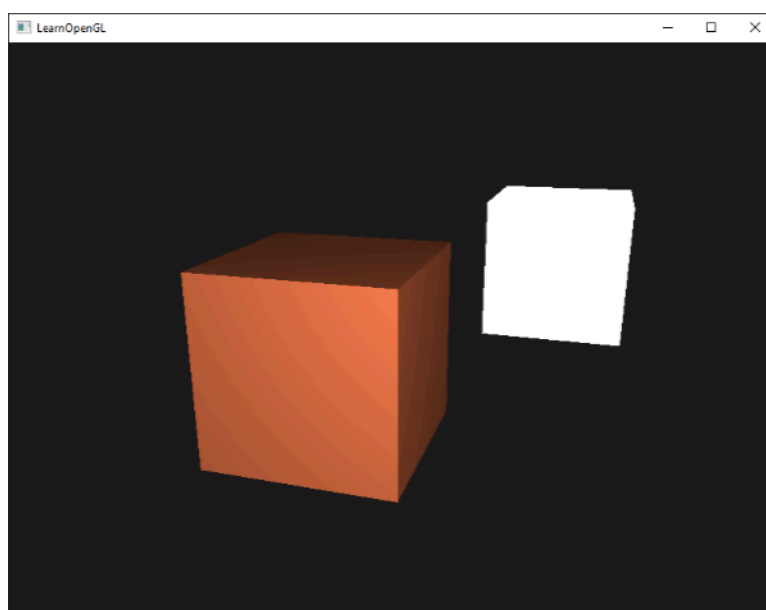


Figura 15: Representação luz difusa

Especular:

Ao contrário da difusa que é refletida em todas as direções, esta possui uma única direção de incidência, porém também será refletida em uma única posição.

- Gera pontos de brilho no objeto
 - O brilho dependerá da posição do observador
- O brilho será da cor da luz
- Seu brilho será emitido em todas as direções
- Precisamos determinar:
 - A componente ambiente da luz (opcional)
 - as coordenadas da luz (x,y,z)
 - a cor da luz especular dada pelos canais RGB
 - a cor da luz difusa dada pelos canais RGB
 - a cor especular do objeto dada também pelos canais RGB
 - a capacidade de brilho do objeto (semelhante ao quão polido o objeto será)

Para inserimos uma luz especular no OpenGL precisamos então:

1. Definir a cor da luz ambiente, posição e cor para a luz difusa e especular (usando constantes)

```
const
LuzAmbiente : Array[0..3] of GLfloat = (0.5, 0.5, 0.5, 1.0);
LuzDifusa : Array[0..3] of GLfloat = (0.4, 0.4, 0.4, 1.0);
LuzEspecular: Array[0..3] of GLfloat = (1.0, 0.0, 1.0, 1.0);
PosicaoLuz : Array[0..3] of GLfloat = (0.0, 20.0, 60.0, 1.0);
Especularidade: Array[0..3] of GLfloat = (0.0, 0.0, 0.0, 1.0);
```

2. Ativar o uso da componente ambiente da luz e o color tracking

```
// Habilita o uso de iluminação
glEnable(GL_LIGHTING);

// Ativa o uso da luz ambiente
glLightModelfv(GL_LIGHT_MODEL_AMBIENT, @LuzAmbiente);

// Ativa o "Color Tracking"
glEnable(GL_COLOR_MATERIAL);
```

3. Definir e ativar a luz especular

```
// Define os parâmetros da Luz número Zero
glLightfv(GL_LIGHT0, GL_AMBIENT, @LuzAmbiente);
glLightfv(GL_LIGHT0, GL_DIFFUSE, @LuzDifusa );
glLightfv(GL_LIGHT0, GL_SPECULAR, @LuzEspecular );
glLightfv(GL_LIGHT0, GL_POSITION, @PosicaoLuz );
glEnable(GL_LIGHT0);
```

4. Definir a cor especular do objeto e o seu grau de especularidade

```
// Define a reflectancia do material
glMaterialfv(GL_FRONT, GL_SPECULAR, @Especularidade);

// Define a concentração do brilho.
// Quanto maior o valor do Segundo parametro, mais concentrado será o brilho.
// (Valores válidos: de 0 a 128)
glMateriali(GL_FRONT, GL_SHININESS, 20);
```

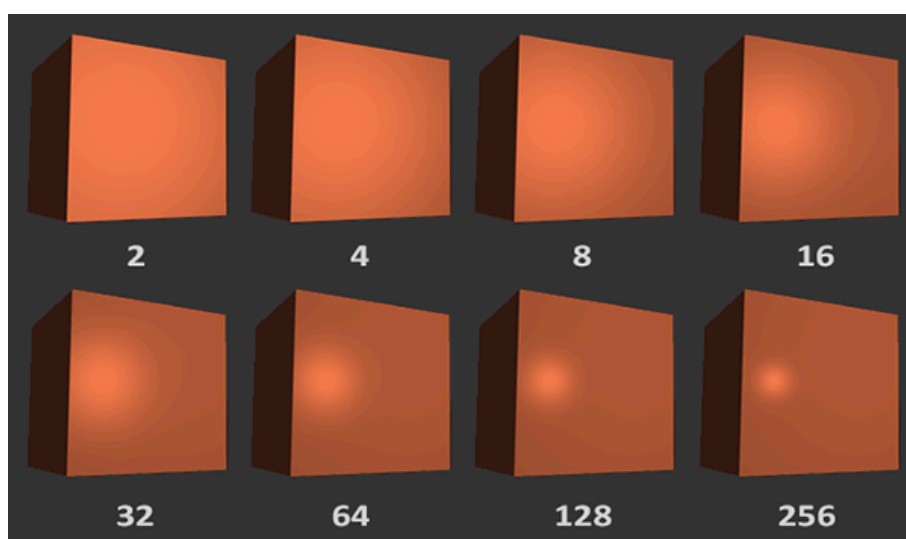


Figura 16: Representação luz especular

Combinação das três componentes:

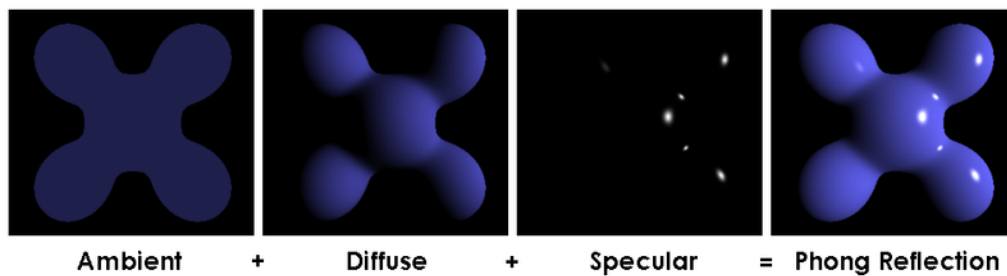


Figura N: Combinação das componentes de luz em um objeto 3D no OpenGL

Renderização

A renderização é o estágio onde a descrição matemática da cena se torna uma imagem digital. No OpenGL, isso ocorre através do **Pipeline Gráfico**, uma "esteira" de transformações.

Pipeline: Fases do Processo de Realismo Visual

O nível de realismo em uma cena é proporcional ao número de fases aplicadas e ao detalhamento de cada uma. Na computação gráfica, esse processo envolve até sete etapas principais, ajustadas conforme a necessidade de desempenho e custo de processamento.

1. Modelagem

- **O que é:** Construção geométrica do objeto tridimensional, definindo sua forma e estrutura através de vértices, arestas e faces.
- **No OpenGL:** Ocorre através da definição de vetores de dados enviados para a GPU. Utilizamos **VBOs (Vertex Buffer Objects)** para armazenar as coordenadas dos vértices e **VAOs (Vertex Array Objects)** para organizar como esses dados devem ser lidos.

2. Projeção

- **O que é:** Aplicação de transformações lineares para converter as coordenadas 3D do mundo para o espaço 2D da tela, aplicando conceitos de perspectiva (objetos distantes parecem menores).
- **No OpenGL:** É realizada através das matrizes **MVP (Model-View-Projection)**. A matriz de Projeção define o volume de visão (*frustum*) e transforma as coordenadas para o espaço de recorte (*Clip Space*).

3. Backface Culling

- **O que é:** Eliminação de faces que estão voltadas para o lado oposto ao observador. Se o verso de um polígono não é visível, ele não precisa ser processado.
- **No OpenGL:** Ativado via `glEnable(GL_CULL_FACE)`. O OpenGL determina o que é "frente" ou "verso" baseando-se na ordem de declaração dos vértices (sentido horário ou anti-horário).

4. Clipping (Recorte)

- **O que é:** Descarte de partes da geometria que estão fora do campo de visão da câmera ou além dos planos de corte (muito perto ou muito longe).
- **No OpenGL:** Ocorre automaticamente após a projeção. O hardware descarta primitivas fora das coordenadas normalizadas do dispositivo. Um efeito comum em jogos é o recorte do plano *Near*, onde objetos "somem" ao encostarmos neles.

5. Rasterização

- **O que é:** Conversão da representação matemática (vetores) em pixels para o dispositivo de exibição.
- **No OpenGL:** O estágio onde os objetos são preenchidos por fragmentos. Aqui, o OpenGL **interpola os atributos** dos vértices (como cores e normais) para cada pixel interno, garantindo transições suaves.

6. Remoção de Superfícies Ocultas (HSR)

- **O que é:** Diferente do Culling (que descarta faces inteiras), esta fase atua após a rasterização, operando pixel a pixel para decidir qual objeto está à frente de outro em caso de sobreposição.
- **No OpenGL:** Utiliza-se o **Z-Buffer (Depth Buffer)**. Ativado por `glEnable(GL_DEPTH_TEST)`, o sistema compara a profundidade de cada novo pixel com o valor já armazenado; se o novo pixel estiver mais perto, ele substitui o anterior.

7. Sombreamento (Shading)

- **O que é:** A etapa final de coloração, onde são calculados os efeitos de iluminação, texturas, transparência e reflexos para cada pixel individualmente.
- **No OpenGL:** É executado no **Fragment Shader**. Aqui aplicamos modelos físicos (como o modelo de Phong) para calcular como a luz interage com a normal da superfície e a posição do observador para gerar a cor final.

Software: Realismo por Passadas (Multipass Rendering)

A renderização por passadas é o padrão da computação gráfica profissional. Em vez de calcular toda a imagem de uma vez, cada atributo da cena é renderizado separadamente em "camadas". Isso economiza memória e permite modificações pontuais (como ajustar o brilho ou a intensidade de uma sombra) sem precisar refazer todo o processo de renderização.

No **OpenGL**, o mecanismo que viabiliza esse fluxo é o **FBO (Framebuffer Object)**. Ele permite desviar a saída da renderização da tela para texturas na memória, que depois são combinadas no resultado final.

1. Cores (Diffuse Pass)

- **O que é:** Atribuição das cores base e da iluminação ambiente da cena.
- **No OpenGL:** Renderiza os objetos apenas com suas texturas e cores sólidas, sem tratar de reflexões ou brilhos intensos. É a "base" sobre a qual as outras passadas serão somadas.

2. Brilho (Highlight / Specular Pass)

- **O que é:** Adição de luminosidade direcional e brilhos intensos (pontos de luz que refletem no material).
- **No OpenGL:** Isola a componente especular do modelo de iluminação. O resultado mostra os objetos bem iluminados, mas geralmente sobre um fundo sem sombras, focando apenas no brilho do material.

3. Reflexões (Reflection Pass)

- **O que é:** Adição de reflexos a superfícies como espelhos, pisos encerados e água.
- **No OpenGL:** Geralmente utiliza técnicas de *Environment Mapping* (Cubemaps) ou renderiza a cena de uma perspectiva invertida para gerar a textura do reflexo que será aplicada sobre a superfície.

4. Iluminações (Light Pass)

- **O que é:** Renderização individual de cada fonte de luz (lâmpadas, velas, raios) e sua influência específica na cena.
- **No OpenGL:** Permite controlar o acendimento progressivo de luzes ou efeitos de faróis de forma independente. Cada luz pode ser "somada" à imagem final através de cálculos de *blending* (mistura de cores).

5. Sombras (Shadow Pass)

- **O que é:** Renderização das sombras projetadas pelos objetos no solo ou sobre outros elementos.
- **No OpenGL:** Utiliza a técnica de **Shadow Mapping**, onde a cena é renderizada do ponto de vista da luz para gerar um mapa de profundidade, determinando quais pixels estão "escondidos" da fonte luminosa.

6. Efeitos Especiais (FX Pass)

- **O que é:** Renderização de elementos atmosféricos ou dinâmicos, como explosões, fumaça e distorções de lente.
- **No OpenGL:** Muitas vezes envolve sistemas de partículas ou shaders de pós-processamento que operam sobre a imagem já rasterizada, podendo ser adicionados de forma independente.

7. Profundidade (Depth Pass)

- **O que é:** Adição artificial da noção de profundidade para quebrar a nitidez excessiva das imagens sintéticas.
- **No OpenGL:** O sistema gera um mapa de profundidade (baseado no Z-buffer) e aplica algoritmos de **Blurring** (desfoque). Isso faz com que objetos distantes fiquem desfocados, simulando o foco de uma lente real e gerando sensação de distância.

Vantagem Técnica:

Utilizando FBOs para cada uma dessas passadas, o desenvolvedor ganha controle total sobre a composição final. Por exemplo, é possível aumentar a opacidade apenas da **Shadow Pass** para tornar as sombras mais dramáticas sem alterar a cor base (**Diffuse Pass**) do objeto.

Toon Shading (Implementação NPR)

Diferente do degradê suave de Phong, o Toon Shading no OpenGL funciona por **quantização** da luz.

Lógica do Shader:

- Em vez de usar o valor direto do produto escalar entre a normal e a luz, criamos faixas fixas:
 - Se a intensidade for alta (> 0.8), use cor clara.
 - Se for média (0.4 a 0.8), use cor média.
 - Se for baixa (< 0.4), use cor escura (sombra sólida).]

Além da quantização da luz, seu visual inclui um **contorno escuro** nas bordas do objeto. No OpenGL, isso é feito em uma segunda passada: o objeto é renderizado levemente ampliado com as faces frontais descartadas e cor preta, de forma que apenas o volume expandido da silhueta apareça atrás do objeto original.

Combinando a quantização com esse contorno, o resultado simula o traço de um desenho animado.

Hiper-Realismo e Pós Processamento

O hiper-realismo no OpenGL é alcançado manipulando os buffers de cor e profundidade após a renderização principal (pós-processamento).

Técnicas de Hiper-realismo

A seguir se encontram, de maneira simplificada, as técnicas usadas para aplicar um efeito de Hiper-realismo nas imagens e a teoria por trás da sua implementação no OpenGL. (

1. HDRI / IBL (Image Based Lighting)

- **O que é:** Uso de imagens de fundo (Cubemaps) com informações de iluminação real para substituir fontes de luz artificiais. Isso resolve o problema de cenários distantes (céu, nuvens) que não são modelados mas que devem influenciar a luz da cena.
- **No OpenGL:** No pipeline moderno (IBL), o mapa HDR é pré-processado em duas versões: uma **difusa** (luz ambiente distribuída) e uma **especular** (reflexos direcionais). O Fragment Shader amostra essas texturas com base na normal e no vetor de reflexão, permitindo que o brilho do sol na imagem HDR ilumine o objeto automaticamente.

2. Atenuação Atmosférica

- **O que é:** Simulação do efeito de partículas suspensas (poeira, vapor, fumaça) que dispersam a luz, tornando objetos distantes mais foscos e desfocados.
- **No OpenGL:** Aplicado em pós-processamento usando o mapa de profundidade (Depth Pass). O shader mistura a cor original do pixel com a cor da neblina conforme a distância aumenta, reforçando a sensação de escala do ambiente.

3. Area Light + Soft Shadows

- **O que é:** Simulação de fontes de luz que possuem área (como janelas ou softboxes), em vez de serem apenas pontos matemáticos. Isso gera sombras suaves que se dispersam com a distância.
- **No OpenGL:** Simula o efeito de iluminação global sem o custo do Ray Tracing. O sistema realiza múltiplas amostragens (samples) ao redor da fonte de luz para calcular a penumbra, resultando em um sombreamento muito mais convincente.

4. Ambient Occlusion (SSAO)

- **O que é:** Simulação do escurecimento natural em frestas e cantos onde a luz ambiente não consegue chegar facilmente.
- **No OpenGL:** Opera em pós-processamento sobre o **Depth Buffer**. O algoritmo (SSAO, HBAO ou HDAO) verifica o quão "bloqueado" está o hemisfério acima de cada pixel; quanto mais geometria vizinha houver, mais escuro o pixel será pintado.

5. Sub-Surface Scattering (SSS)

- **O que é:** Simulação da luz que penetra em materiais translúcidos (pele, cera, folhas), se dispersa internamente e sai em um ponto diferente.
- **No OpenGL:** Utiliza a função **BSSRDF** para calcular o trajeto interno da luz. Tecnicamente, o shader mede a espessura do objeto; áreas mais finas (como orelhas contra a luz) recebem uma tonalidade avermelhada transmitida, simulando o espalhamento interno.

6. Depth of Field (DoF)

- **O que é:** Simulação do foco de lentes reais, onde apenas uma faixa de profundidade é nítida e o restante é borrado.
- **No OpenGL:** O shader utiliza o Depth Buffer para aplicar um filtro de *blur* (desfoque) seletivo nos pixels que estão fora da distância de foco definida pela câmera virtual.

7. Motion Blur

- **O que é:** Desfoque causado pelo movimento, simulando o tempo de exposição do filme fotográfico.
- **No OpenGL:** Implementado através de um **Velocity Buffer**, que calcula a posição do objeto entre o quadro anterior e o atual. O shader mistura as cores ao longo dessa trajetória de movimento para criar a fluidez visual.

8. Film Grain e Lens Flare

- **O que é:** Adição de "imperfeições" ópticas. O **Film Grain** adiciona um ruído sutil para reduzir o aspecto "limpo" demais do digital. O **Lens Flare** simula reflexos de luz dentro das lentes da câmera.
- **No OpenGL:** São filtros de fragmento aplicados na última etapa do pipeline. O ruído é gerado por algoritmos de dispersão aleatória, enquanto os flares são sprites ou formas geométricas renderizadas sobre fontes de luz intensas.

9. Glow (Glow Effect) / Bloom

- **O que é:** Adição de uma aura luminosa ao redor de objetos com alta emissão de luz.
- **No OpenGL:** Utilizado em partículas de explosão, exaustão de turbinas ou estrelas. O shader identifica áreas com brilho acima de um certo limite e aplica um desfoque radial expandido (Gaussian Blur) que é somado à imagem original.

Referências

- <https://panda.ime.usp.br/introcg/static/introcg/03-sistemasGraficos.html>
- <https://pt.wikipedia.org/wiki/OpenGL>
- <https://www.inf.pucrs.br/~flash/cg/aulas/modelagem/>
- <https://learnopengl.com/Introduction>
- <https://www.tecgraf.puc-rio.br/ftp/pub/lfm/CIV2802-Cor-OpenGL.pdf>
- <https://learnopengl.com/Lighting/Colors>
- <https://docs.gl/gl4/glClearColor>
- <https://learnopengl.com/Getting-started/Transformations>
- <https://codesignal.com/learn/courses/3d-worlds-and-matrix-transformations/lessons/implementing-glm-transformations>
- <https://learnopengl.com/Getting-started/Coordinate-Systems>
- <http://www.opengl-tutorial.org/beginners-tutorials/tutorial-3-matrices/>
- <https://math.hws.edu/graphicsbook/c2/s3.html>
- <https://www.inf.pucrs.br/~manssour/OpenGL/Iluminacao.html>
- <https://www.inf.pucrs.br/pinho/CG/Aulas/OpenGL/LuzesGL.html>
- <https://learnopengl.com/Lighting/Basic-Lighting>
- <https://registry.khronos.org/OpenGL-Refpages/gl2.1/xhtml/gLight.xml>
- <https://learnopengl.com/Advanced-OpenGL/Framebuffers>
- <https://learnopengl.com/Advanced-OpenGL/Face-culling>
- <https://learnopengl.com/Advanced-Lighting/SSAO>
- https://www.reddit.com/r/gamedev/comments/1h3ia86/tutorial_realtime_subsurface_scattering_in_opengl/
- <https://www.opengl.org/archives/resources/features/KilgardTechniques/LensFlare/>
- <https://learnopengl.com/Advanced-Lighting/Bloom>
- <https://learnopengl.com/Guest-Articles/2022/Area-Lights>
- <https://www.opengl.org/archives/resources/code/samples/advanced/advanced96/node40.html>
- <https://community.khronos.org/t/film-grain-in-opengl/36953>
- <https://learnopengl.com/Advanced-OpenGL/Depth-testing>

- <https://learnopengl.com/Advanced-Lighting/HDR>
- <https://learnopengl.com/PBR/IBL/Diffuse-irradiance>
- <https://developer.nvidia.com/gpugems/gpugems/part-iii-materials/chapter-16-real-time-approximations-subsurface-scattering>
- https://en.wikibooks.org/wiki/OpenGL_Programming/Depth_of_Field
- https://www.cs.rpi.edu/~cutler/classes/advancedgraphics/S12/final_projects/hutchins_kim.pdf
- <https://www.openglprojects.in/2011/05/motion-blur.html#gsc.tab=0>
- <https://gamedev.stackexchange.com/questions/44678/suitability-of-ground-fog-using-layered-alpha-quads>
- <https://stackoverflow.com/questions/16545334/how-to-get-the-depth-of-field-effect-using-opengl>